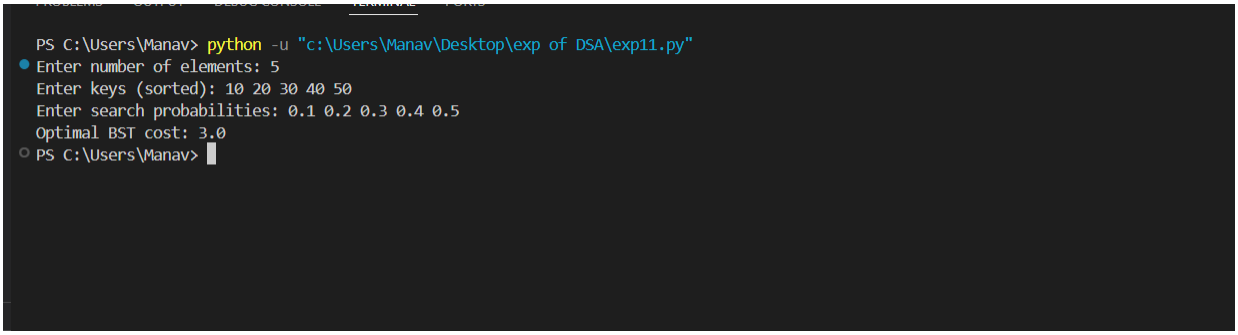
Name: Manav Uttekar
Rollno:71
Problem Statement: Given sequence k = k1 <k2 < … <kn of n sorted keys, with a search probability pi for each key
ki . Build the Binary search tree that has the least search cost given the access probability for each
key?

```
import sys

def optimal_bst(keys, freq, n):
    cost = [[0 for _ in range(n)] for _ in range(n)]

    for i in range(n):
        cost[i][i] = freq[i]

    for L in range(2, n + 1):  # L is the chain length
        for i in range(n - L + 1):
            j = i + L - 1
            cost[i][j] = sys.maxsize

            # Try making all keys in interval keys[i..j] as root
            for r in range(i, j + 1):
                c = (cost[i][r - 1] if r > i else 0) + \
                    (cost[r + 1][j] if r < j else 0) + \
                    sum(freq[i:j + 1])

                if c < cost[i][j]:
                    cost[i][j] = c
    return cost[0][n - 1]

def main():
    n = int(input("Enter number of elements: "))
    keys = list(map(int, input("Enter keys (sorted): ").split()))
    freq = list(map(float, input("Enter search probabilities:
").split()))

    if len(keys) != n or len(freq) != n:
        print("Error: Number of keys and probabilities must match the
specified count.")
        return

    min_cost = optimal_bst(keys, freq, n)
    print(f"Optimal BST cost: {min_cost}")

if __name__ == "__main__":
    main()
```

OUTPUT :


PS C:\Users\Manav> python -u "c:\Users\Manav\Desktop\exp of DSA\exp11.py"
Enter number of elements: 5
Enter keys (sorted): 10 20 30 40 50
Enter search probabilities: 0.1 0.2 0.3 0.4 0.5
Optimal BST cost: 3.0
PS C:\Users\Manav>